

Marathi Text-to-Speech Adaptation of Indic-Speak Using QLoRA

A Project Report on Parameter-Efficient Fine-Tuning with the AI4Bharat Rasa Dataset

Task: Marathi text-to-speech generation **Base model:** bodhan-ai/indic-speak **Dataset:** ai4bharat/Rasa, Marathi configuration **Adaptation method:** 4-bit QLoRA **Audio representation:** SNAC hierarchical codec tokens **Decoder:** Vocos

Abstract

This project adapts the multilingual Indic-Speak text-to-speech model to Marathi using Quantized Low-Rank Adaptation (QLoRA). The objective was to improve Marathi-specific audio-token modeling without fully fine-tuning the base language model or the neural audio components. The official Marathi portion of the AI4Bharat Rasa dataset was used, comprising 26,960 training examples and 2,995 held-out test examples. Audio was converted to mono, resampled to 24 kHz when required, and encoded with a frozen SNAC codec. The resulting three hierarchical codec levels were serialized into the seven-token-per-frame representation expected by Indic-Speak. Prompts combined Marathi text, speaking style, gender-derived speaker conditioning, and target audio tokens.

The Indic-Speak language model was loaded with 4-bit NF4 quantization. LoRA adapters with rank 16, alpha 32, and dropout 0.05 were trained in the attention and multilayer-perceptron projection modules. Only 24,313,856 parameters, or 0.7312% of the model, were trainable. One epoch required 3,370 optimizer steps and approximately 2 hours 40 minutes on an NVIDIA H200. The final training loss was 3.8043 and the held-out test loss was 3.7393. These values measure teacher-forced audio-token prediction and do not independently establish perceptual speech quality. Generated baseline/fine-tuned comparisons confirm that the complete inference path works, but they also expose a runaway-generation failure for one short proper noun. The project therefore demonstrates a reproducible and memory-efficient Marathi adaptation pipeline while identifying perceptual evaluation and generation robustness as the main areas for future work.

Keywords: Marathi TTS, Indic-Speak, QLoRA, LoRA, SNAC, Vocos, parameter-efficient fine-tuning, neural audio codec

1. Introduction

Text-to-speech systems for high-resource languages benefit from large speech corpora, mature pretrained models, and extensive evaluation infrastructure. Marathi has fewer openly available, carefully labeled resources and fewer pretrained systems than English. Training a modern TTS model from scratch would also require substantial data and compute.

This project addresses that constraint by adapting an existing multilingual speech-generation model instead of retraining an entire model. Indic-Speak already provides a causal language-model formulation for speech generation. It represents speech as discrete neural audio tokens and predicts those tokens from a structured

text prompt. This makes parameter-efficient fine-tuning a practical way to specialize the model for Marathi.

The work was designed around four goals:

1. Preserve the capabilities of the pretrained Indic-Speak model while specializing it for Marathi.
2. Keep GPU memory and training time manageable through 4-bit quantization and LoRA adapters.
3. retain all valid examples, speaking styles, and both gender groups in the official Rasa splits.
4. produce a revision-pinned and auditable experiment with saved preprocessing, training, and inference artifacts.

The central research question is:

Can the Indic-Speak language model be adapted to Marathi audio-token prediction using a small trainable parameter set while keeping the base model and audio stack frozen?

2. Complete System Workflow



Figure 1. Complete training and inference workflow. The upper lane shows Rasa ingestion, audio normalization, SNAC tokenization, prompt construction, and QLoRA training. The trained PEFT adapter is then attached to the frozen Indic-Speak model in the lower inference lane. Generated audio tokens are reconstructed as SNAC codes and decoded into a 24 kHz Marathi waveform by Vocos.

The architecture deliberately separates training from inference:

- During training, speech examples provide the target audio-token sequences used to optimize the LoRA parameters.

- During inference, only Marathi text and conditioning metadata are supplied. The adapted language model generates audio tokens autoregressively, which are converted back to audio.
- Indic-Speak, the SNAC codec, and Vocos remain frozen. The Marathi LoRA adapter is the only learned project-specific model component.

3. Data

3.1 Dataset selection

The Marathi configuration of AI4Bharat Rasa was selected because it provides paired speech and text together with gender and style metadata. The official split definitions were retained rather than creating a random project-specific split. This keeps the experiment reproducible and avoids accidental leakage introduced by a new splitting procedure.

Split	Examples	Transcript words	SNAC frames
Train	26,960	305,293	2,116,724
Test	2,995	33,840	233,611

No arbitrary duration, style, gender, row-count, or token-length filtering was applied. Keeping the complete official splits avoids silently narrowing the target domain, although it also means the training process must handle long sequences and fertility outliers.

3.2 Conditioning information

Each example contributes:

- a Marathi transcript;
- a speech waveform;
- a speaking-style label;
- a gender label used to derive a model speaker label.

The Rasa dataset does not provide stable speaker identifiers. The following proxy labels were therefore used:

Dataset label	Indic-Speak conditioning label
MALE or M	MarathiRasaMale
FEMALE or F	MarathiRasaFemale

These labels represent broad gender groups and must not be interpreted as verified individual speaker identities. This choice made it possible to preserve a speaker-conditioning field compatible with the Indic-Speak prompt format without inventing identities unsupported by the dataset.

3.3 Split-overlap audit

The training and test manifests were compared using filenames, decoded-audio SHA-256 hashes, and exact transcript strings.

Overlap type	Count	Interpretation
Filename overlap	0	No repeated source filenames across splits
Decoded-audio hash overlap	0	No byte-equivalent decoded waveforms across splits
Exact transcript overlap	825	Some sentences appear in both splits with different recordings

The absence of filename and audio-hash overlap is reassuring. However, 825 shared transcript strings mean the test loss is not a fully text-disjoint evaluation. The recordings are different, but familiarity with the same text may make the held-out token-prediction task less difficult than evaluation on completely unseen sentences.

4. Methodology

4.1 Reproducibility and revision pinning

Both the model and dataset revisions were fixed before processing:

Indic-Speak revision: `76e0814f189321efe550011850cd288dae366aad`

Rasa revision: `632f55c7ac590219d41cd7adffce5b440e4604f5`

Random seed: `3407`

Pinning revisions prevents later upstream changes from silently altering model weights, dataset rows, or metadata. The full environment is preserved in `requirements.lock.txt`, while the experiment settings are recorded in `run_config.json`.

4.2 Audio normalization

Every waveform was decoded and converted to a consistent representation:

1. Load the source audio associated with a Rasa row.
2. Convert multichannel input to mono when necessary.
3. Resample the waveform to 24 kHz when the source sampling rate differs.
4. pass the normalized waveform to the frozen SNAC encoder.

The 24 kHz target rate was chosen because it matches the codec and decoding path used by Indic-Speak. Performing normalization before tokenization prevents inconsistent sample rates or channel layouts from changing the meaning and density of the target token sequence.

4.3 SNAC audio tokenization

SNAC is a hierarchical neural audio codec. It converts the waveform into three discrete codec levels with different temporal resolutions. Indic-Speak expects these codes in a seven-token-per-frame layout, so the three levels were packed into the required serialization pattern.

For a clip with (F) base SNAC frames, the serialized audio portion contributes approximately (7F) codec tokens, plus control or termination tokens introduced by the model format. The codec was frozen because the aim was to adapt linguistic and acoustic token prediction, not to retrain the speech representation itself.

Preprocessing fertility was defined as:

$$\text{fertility} = \frac{\text{base SNAC frames}}{\text{whitespace-delimited transcript words}}$$

Split	Mean	Median	P05	P95
Train	7.6020	7.0	4.8333	12.5000
Test	7.5799	7.0	4.8967	12.3333

The similar distributions indicate that the train and test splits have comparable audio-token density. Whitespace-delimited words are only an approximation for Marathi, so fertility is treated as a preprocessing diagnostic rather than a linguistic or quality metric.

4.4 Prompt construction

The official Indic-Speak prompt builder was used. Each training sequence combines:

Marathi text;
speaking style;
gender-derived speaker conditioning;
serialized target audio tokens;
an end-of-speech or termination region.

The prompt portion was masked in the labels. Loss was calculated only on the audio-token and termination-token region. This is important because the goal is to train the model to predict speech tokens conditioned on a supplied prompt, not to spend optimization capacity reproducing the prompt text itself.

For token labels (y_t), model parameters (θ), and the first target-audio position (T_p), the training objective can be expressed as:

$$\mathcal{L}(\theta) = -\sum_{t=T_p}^T \log p_{\theta}(y_t \mid y_{<t})$$

Positions before (T_p) receive the ignore label and therefore do not contribute to cross-entropy.

4.5 QLoRA adaptation

The base causal language model was loaded using 4-bit NormalFloat (NF4) quantization with double quantization. The original weights remained frozen. Small trainable low-rank matrices were inserted into the following projection modules:

- `q_proj`, `k_proj`, `v_proj`, and `o_proj` in attention blocks;
- `gate_proj`, `up_proj`, and `down_proj` in feed-forward blocks.

For an original frozen weight matrix (W), LoRA models the adapted transformation as:

$$W' = W + \frac{\alpha}{r}BA$$

where (A) and (B) are trainable low-rank matrices, ($r=16$), and ($\alpha=32$). This reduces the number of trainable parameters while still allowing changes throughout both attention and feed-forward computation.

QLoRA setting	Value
Quantization	4-bit NF4 with double quantization
Compute type	torch.bfloat16
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
Trainable parameters	24,313,856
Trainable fraction	0.7312%
Bias adaptation	None

4.6 Why QLoRA was chosen

Full-model fine-tuning was not necessary for this project and would have increased memory use, checkpoint size, and the risk of overwriting useful multilingual representations. QLoRA was selected for three reasons:

1. **Memory efficiency.** Four-bit frozen base weights reduce the largest persistent memory cost.
2. **Storage efficiency.** The final deliverable is a compact adapter rather than a duplicate of the complete base model.
3. **Controlled specialization.** Low-rank updates allow Marathi adaptation while leaving the pretrained base weights intact.

The tradeoff is that adapter capacity is restricted. If the target-domain shift is too large, a low-rank update may underfit relative to full fine-tuning. The one-epoch experiment evaluates feasibility rather than establishing the maximum achievable quality.

4.7 Optimization configuration

Setting	Value
Epochs	1
Optimizer steps	3,370
Micro-batch size	1
Gradient accumulation	8
Effective batch size	8 examples
Learning rate	1e-4
Optimizer	Paged AdamW 8-bit
Scheduler	Cosine
Warmup steps	170
Gradient checkpointing	Enabled
Maximum sequence length	No arbitrary cutoff

A micro-batch of one was used because speech-token sequences vary substantially in length and can be large. Gradient accumulation preserved an effective batch size of eight without requiring all examples to be resident in GPU memory simultaneously. Gradient checkpointing traded additional computation for reduced activation memory.

4.8 Evaluation procedure

After completing the single training epoch, the model was evaluated once on the official test split using teacher forcing. This produced a token-level cross-entropy loss over the audio and termination tokens.

The adapter was also reloaded into the official Indic-Speak TTS implementation. Baseline and fine-tuned samples used the same prompt, conditioning labels, random seed, temperature of 0.6, and top-p value of 0.9. Holding these settings constant makes the paired outputs suitable for controlled listening comparisons, although the small number of samples is not sufficient for a statistical quality claim.

5. Inference Workflow

Inference uses the following sequence:

Receive Marathi text, style, and a gender-derived speaker label.

Build the official Indic-Speak prompt.

Load the pinned frozen Indic-Speak base language model.

Attach the trained Marathi PEFT adapter to the language-model component.

Generate serialized audio tokens autoregressively.

Detect the end-of-speech token or stop at the configured generation limit.

Convert the seven-token layout back into the three SNAC codec levels.

Decode the reconstructed representation through the official Vocos path.

Save the resulting waveform as 24 kHz WAV audio.

The adapter cannot independently synthesize audio. It must be combined with the compatible base model and the same prompt, token-conversion, and decoder logic used by Indic-Speak.

6. Experimental Results

6.1 Training and resource results

Measurement	Result
Training examples	26,960
Test examples	2,995
Epochs	1
Optimizer steps	3,370
Training loss	3.8043
Test loss	3.7393
Runtime	9,594.42 seconds (~2 h 40 min)
Throughput	2.81 samples/second
Training steps/second	0.351
Peak allocated GPU memory	12.43 GiB
Hardware	NVIDIA H200
Approximate total FLOPs	(2.844×10^{17})

The slightly lower test loss than training loss is not by itself evidence that the model generalizes better on test data. Training loss is accumulated while dropout is active and parameters are changing, whereas test loss is measured after training with evaluation behavior. The presence of 825 repeated transcript strings may also reduce the effective novelty of the test set.

6.2 Interpretation of token loss

Cross-entropy indicates how well the model predicts the reference audio-token sequence under teacher forcing. It is useful for checking whether optimization is functioning and comparing otherwise matched experiments. It does not directly measure:

- pronunciation accuracy;
- intelligibility;
- naturalness or prosody;
- speaker similarity;
- style consistency;

- hallucinated, repeated, or omitted words;
- robustness during free autoregressive generation.

Therefore, no perceptual-quality improvement is claimed solely from the recorded losses.

6.3 Generated comparisons

Four baseline/fine-tuned pairs were created: three from held-out test prompts and one manually added neutral Marathi prompt. Three fine-tuned generations completed with plausible durations relative to their baselines. One one-word `PROPER NOUN` example did not emit the expected end-of-speech token and reached `max_new_tokens=2520`, producing a 30.72-second waveform. The corresponding baseline was only 0.60 seconds.

This example is treated as a failure rather than as a valid longer pronunciation. It demonstrates why teacher-forced test loss must be supplemented with free-generation evaluation.

7. Challenges and How They Were Addressed

7.1 Variable-length speech sequences

Challenge. Audio-token sequences are much longer than ordinary text labels, and their lengths vary with utterance duration and speaking rate. Long examples can cause abrupt memory peaks.

Response. The run used micro-batches of one, gradient accumulation of eight, 4-bit base weights, paged 8-bit optimization, and gradient checkpointing. No arbitrary sequence-length filter was introduced, preserving the official dataset while controlling memory use.

7.2 Correct hierarchical token packing

Challenge. SNAC produces three codec levels with different temporal structures, while Indic-Speak expects a particular seven-token-per-frame ordering. Incorrect interleaving would create syntactically valid tensors that decode incorrectly.

Response. The project used Indic-Speak-compatible serialization and recorded per-example frame and token counts in the manifest. Fertility summaries were then used as a corpus-level sanity check.

7.3 Loss masking

Challenge. A concatenated prompt-and-audio sequence can accidentally train the model to predict the input prompt. This dilutes the desired audio-token objective.

Response. Prompt positions were assigned ignore labels so that cross-entropy was computed only over audio and termination targets.

7.4 Missing stable speaker identities

Challenge. Rasa provides gender metadata but no stable speaker IDs, while Indic-Speak expects speaker conditioning.

Response. Two explicit Marathi Rasa conditioning labels were derived from gender. The report documents that these are group proxies, not individual speaker identities, to prevent overinterpretation.

7.5 Test-set interpretation

Challenge. Exact transcript overlap can make a held-out split less independent even when the recordings are different.

Response. Filename, audio-hash, and transcript overlap were audited separately. The 825 repeated transcripts are disclosed as an evaluation limitation rather than removed after observing the test data.

7.6 Runaway generation

Challenge. One short fine-tuned prompt failed to generate a timely end-of-speech token and continued until the maximum token limit.

Response. The output was flagged as a failure. Future inference should include stronger stopping rules, duration or token-length guards based on transcript length, repetition detection, and a broader short-utterance evaluation set.

7.7 Reproducibility across changing dependencies

Challenge. Model code, dataset contents, and library APIs may change over time.

Response. Model and dataset revisions were pinned, the Python environment was captured, the random seed was recorded, and artifacts such as the manifest, preprocessing statistics, training history, adapter configuration, and comparison metadata were saved.

8. Design Choices and Tradeoffs

Choice	Reason	Tradeoff
Adapt Indic-Speak instead of training from scratch	Reuse multilingual speech-generation knowledge	Inherits assumptions and limitations of the base model
QLoRA with a 4-bit base	Reduce memory and trainable checkpoint size	Less adaptation capacity than full fine-tuning
Freeze SNAC and Vocos	Stabilize the audio representation and decoder	Codec or decoder errors cannot be corrected by training
Use official Rasa splits	Reproducible evaluation	Test set is not transcript-disjoint
Retain all valid rows	Avoid subjective filtering and domain narrowing	Includes long examples and fertility outliers
Use gender-derived speaker labels	Preserve required conditioning without fabricating speaker IDs	Does not provide stable speaker identity
Train for one epoch	Establish a bounded, affordable baseline	Does not determine the best convergence point
Evaluate token loss plus paired audio	Cover optimization and free generation	No formal perceptual score or listening-study statistics

9. Limitations

1. The model was trained for only one epoch, so the experiment does not establish an optimal training duration.
2. Test loss is a teacher-forced token metric and cannot substitute for perceptual evaluation.
3. The official splits contain 825 exact transcript overlaps.
4. The base model may already have encountered related Marathi data; its pretraining corpus overlap is unknown.
5. Gender conditioning is not a stable speaker identity.
6. The listening comparison contains only four prompt pairs.
7. At least one short prompt caused runaway generation.
8. No ASR-based intelligibility metric, mean-opinion-score study, or statistically powered preference test was completed.
9. Fertility uses whitespace-delimited words, which is only an approximate segmentation for Marathi.
10. The measured 12.43 GiB peak allocation does not include every possible runtime or framework memory requirement; practical hardware needs may be higher.

10. Future Work

The highest-priority next steps are:

Run a Marathi listening study comparing baseline and fine-tuned outputs using naturalness, intelligibility, pronunciation, and preference ratings.

Add Marathi ASR transcription and compute character or word error rates, with careful normalization of Devanagari text.

Evaluate a larger stratified prompt set covering style, gender group, duration, named entities, numerals, and code-mixed text.

Add generation safeguards for missing end-of-speech tokens, repetition loops, and implausible duration.

Compare multiple checkpoints or epochs using both token loss and perceptual metrics.

Create a transcript-disjoint evaluation subset to measure generalization to unseen text.

Test different LoRA ranks, learning rates, and target-module subsets through controlled ablations.

Investigate whether explicit length or duration prediction improves stopping behavior.

11. Reproducibility Guide

11.1 Required access and hardware

- Accept the access conditions for the Indic-Speak model and Rasa dataset on Hugging Face.
- Provide a Hugging Face read token through `HF_TOKEN` or the notebook's hidden prompt.
- Use a Linux or Colab CUDA environment.
- A CUDA GPU with at least 24 GiB is a practical starting point, even though this run recorded 12.43 GiB peak allocated memory.

11.2 Main software versions

Package	Version
PyTorch	2.8.0+cu128
Transformers	5.17.0
PEFT	0.21.0
BitsAndBytes	0.50.2
Datasets	3.6.0
Accelerate	1.15.0
SNAC	1.2.1

The complete captured environment is stored in `requirements.lock.txt`.

11.3 Execution sequence

1. Open `IndicSpeak_Marathi_Rasa_QLoRA_.ipynb`.
2. Install or restore the pinned software environment.
3. Authenticate with Hugging Face.
4. Run the notebook from top to bottom.

5. Verify that all 26,960 train rows and 2,995 test rows are encoded.
6. Train for one epoch using the recorded configuration.
7. Evaluate on the official test split.
8. Save the PEFT adapter and tokenizer.
9. Reload the pinned base model and attach the adapter.
10. Generate matched baseline and fine-tuned samples through the official SNAC/Vocos inference path.

11.4 Important artifacts

Artifact	Purpose
<code>IndicSpeak_Marathi_Rasa_QLoRA_.ipynb</code>	End-to-end implementation
<code>run_config.json</code>	Pinned revisions and experiment configuration
<code>manifest.json</code>	Per-example transcript, hash, duration, token, and fertility data
<code>preprocessing_stats.json</code>	Dataset, fertility, and overlap summaries
<code>adapter/</code>	Final PEFT LoRA adapter and tokenizer
<code>metrics.json</code>	Final runtime, loss, throughput, and memory metrics
<code>training_log.json</code>	Step-level training history
<code>loss.png</code>	Training-loss visualization
<code>audio_comparison.json</code>	Generation settings, prompts, files, and durations
<code>baseline_.wav</code> / <code>finetuned_ .wav</code>	Paired listening samples

12. Conclusion

This project demonstrates an end-to-end, parameter-efficient method for adapting Indic-Speak to Marathi. By freezing the base model and audio stack, quantizing the language model to 4-bit NF4, and training LoRA adapters across attention and feed-forward projections, the experiment updated only 0.7312% of the model parameters. The complete official Rasa Marathi splits were processed successfully, and the training run completed in approximately 2 hours 40 minutes with 12.43 GiB peak allocated GPU memory.

The result is a compact Marathi adapter and a reproducible pipeline from dataset ingestion through 24 kHz waveform generation. The measured token losses show that optimization and held-out evaluation completed successfully, but the observed runaway example illustrates the limits of relying on teacher-forced loss alone. The strongest conclusion is therefore not that perceptual Marathi speech quality has already been proven, but that the technical adaptation pipeline is functional, auditable, and ready for a more rigorous perceptual and robustness evaluation.

References

H. Siuzdak, F. Grötschla, and L. A. Lanzendörfer, “SNAC: Multi-Scale Neural Audio Codec,” *arXiv preprint arXiv:2410.14411*, 2024.

E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *International Conference on Learning Representations*, 2022.

T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” *Advances in Neural Information Processing Systems*, 2023.

Bodhan AI, “Indic-Speak,” Hugging Face model repository.

AI4Bharat, “Rasa,” Hugging Face dataset repository.

Appendix A: Recorded Configuration Summary

Base model:	bodhan-ai/indic-speak
Dataset:	ai4bharat/Rasa (Marathi)
Train/test examples:	26,960 / 2,995
Quantization:	4-bit NF4, double quantization
Compute dtype:	bfloat16
LoRA rank/alpha/dropout:	16 / 32 / 0.05
Learning rate:	1e-4
Micro-batch/accumulation:	1 / 8
Epochs:	1
Optimizer steps:	3,370
GPU:	NVIDIA H200
Seed:	3407
Train/test loss:	3.8043 / 3.7393
Runtime:	9,594.42 seconds
Peak allocated memory:	12.43 GiB